Operational Process Integrity and Governance: The System Locked Protocol (SLP)

1. Introduction: The Constitutional Crisis of Agentic Workflows

The deployment of autonomous agents within repository-resident architectures represents a paradigm shift in software engineering, moving from human-centric DevOps to AI-mediated "OpsOps." In this transition, the preservation of **Process Integrity**—the guarantee that the executed reality matches the specified intent—becomes the paramount challenge. The **Stela** system, governed by its **Policy of Truth (PoT)**, establishes a rigorous constitutional framework designed to mitigate the probabilistic nature of Large Language Models (LLMs) through deterministic file-based state. However, recent feedback from the **Contractor (Guest AI)** layer indicates a systemic fragility: while the structural scaffolding of the repository remains intact, the semantic coherence binding the **Integrator (Lead AI)** to the Contractor is deteriorating.

This report provides an exhaustive analysis of this "governance drift" and defines the **System Locked Protocol (SLP)**, a new architectural standard that solves process integrity through **Template-Based Certification**. This approach transitions the system from manual, text-based verification to an automated, symmetric workflow where both **Intent** (Drafting) and **Proof** (Certification) are governed by immutable schemas.

---

2. Analysis of Process Integrity: The Fracture Between Intent and Reality

2.1 The Staffing Protocol and the Semiotic Gap

The **Stela** architecture is defined by a tripartite staffing protocol outlined in PoT.md Section 4.1. This protocol assigns distinct responsibilities: the **Operator** holds the keys to final decisions; the **Integrator** maintains structural integrity; and the **Contractor** executes logic. However, the Contractor's feedback reveals that the handoff between the Integrator and the Contractor is the primary locus of failure. The Integrator generates a **Dispatch Packet (DP)** within TASK.md. While this DP adheres to the strict structural schemas enforced by tools/lint/dp.sh, the *semantic* content—the actual instructions—remains ambiguous. The Contractor reports that "DPs are presenting high-impact ambiguity," forcing the agent to

interpret intent rather than follow explicit directives.

Research into "Agent Integrity" suggests this is a fundamental vulnerability. Unlike traditional software functions, which fail loudly when inputs are malformed, LLM agents often "fail silently" by hallucinating a plausible interpretation of ambiguous instructions.[1]

2.2 The Illusion of Canon: Structural vs. Semantic Validity

The repository utilizes automated gates like tools/verify.sh and tools/lint/truth.sh. These tools effectively police the *syntax* of the repository. Yet, the Contractor explicitly notes: "Lints enforce structure, but not semantic coherence between DP, OPEN, DUMP, and live diff." This distinction represents a "Semantic Gap." A structural linter can verify that TASK.md contains a "Receipts" section, but it cannot verify that the commands listed there actually prove the work was done.

This gap allows for **Ghost Canon**. The TASK.md file becomes a theoretical document—a plan of what *should* happen—while the repository reflects a divergent reality. Because receipts are "too manual," the Contractor often generates the RESULTS artifact based on expectation rather than capturing raw stdout/stderr. This degrades the PoW.md (Proof of Work) ledger from a cryptographic proof of execution into a "trust-me" narrative.

2.3 The Transaction Boundary Deficit

The most severe finding is the "lack of a single enforced transaction boundary." In the **Stela** system, the "transaction" is smeared across multiple surfaces:

1. **The Intent:** TASK.md (Active Work Surface).
2. **The State:** OPEN and DUMP artifacts (Snapshots).
3. **The Work:** git diff (The Code).
4. **The Proof:** PoW.md (Evidence).

There is no mechanism that locks these elements together. The pow-policing workflow ensures PoW.md is touched, but it cannot ensure that its content accurately reflects the code change. Without an atomic "commit" mechanism that updates all surfaces simultaneously based on a single truth source, the system relies on the fallible "memory" of the Contractor.

---

3. The Solution: Symmetric Workflow (The V-Model)

To resolve these vulnerabilities, we adopt the **System Locked Protocol (SLP)** via **Template-Based Certification**. This architecture leverages the "Universal Template Engine" established in **DP-OPS-0067** to create a symmetric workflow.

3.1 The Concept

Currently, the workflow is asymmetric:

- **Start:** ops/bin/draft + dp.md.tpl → **Structured Intent** (The DP).
- **End:** Manual Copy-Paste → **Unstructured Proof** (The Results).

The SLP creates symmetry:

- **Start:** ops/bin/draft + dp.md.tpl → **Structured Intent**.
- **End:** ops/bin/certify + results.md.tpl → **Structured Proof**.

3.2 The Mechanism

We introduce a new binary, ops/bin/certify. This tool acts as the "Mechanized Witness." It removes the human (or AI) element from the *generation* of the proof, leaving them responsible only for the *execution* of the work.

- **Input:** The active TASK.md.
- **Process:** Executes verification commands, captures git state, and validates scope.
- **Output:** Renders RESULTS.md using ops/bin/template and results.md.tpl.

This ensures that a receipt can *only* exist if the verification actually ran and passed. The Contractor cannot "type" a receipt; they must "run" the certifier.

---

4. Technical Specification: The System Locked Toolchain

The following specifications define the components required to implement the SLP.

4.1 Component 1: The Receipt Template

File: ops/src/surfaces/results.md.tpl

This template defines the immutable schema of a valid verification receipt. It uses the strict slot validation logic of ops/bin/template to ensure no data is missing.

---

template_type: surface

template_id: results

template_version: 1.0.0

requires_slots:

- DP_ID
- GIT_HASH
- TIMESTAMP

- VERIFICATION_LOG
- DIFF_STAT
- ALLOWLIST_CHECK

{{DP_ID}} RESULTS

1. Certification Metadata

- **Commit:** {{GIT_HASH}}
- **Timestamp:** {{TIMESTAMP}}
- **Status:** CERTIFIED

2. Scope Verification

{{ALLOWLIST_CHECK}}

3. State Impactbash

{{DIFF_STAT}}

4. Execution Proofs
The following verification commands were executed by `ops/bin/certify`.

```bash
{{VERIFICATION_LOG}}
```

5. Mandatory Closing Block

{{CLOSING_BLOCK}}

4.2 Component 2: The Certifier Binary
File: `ops/bin/certify`

This script serves as the transaction coordinator. It binds the Intent (`TASK.md`) to the Proof (`RESULTS.md`).

Logic Flow:
1. **Parse:** Extract `DP_ID` and the `Target Files allowlist` from `TASK.md`.
2. **Integrity Check:** Run `tools/lint/integrity.sh` (see 4.3). If this fails (scope violation), the certifier aborts. No receipt is generated.
3. **Execute:** Parse the `Verification commands` list from `TASK.md`. Run each command in a subshell, capturing `stdout`, `stderr`, and exit codes into a `VERIFICATION_LOG` buffer.
4. **Capture:** Run `git diff --stat` and `git diff --name-only` to populate `DIFF_STAT`.
5. **Render:** Call `ops/bin/template render results` passing all captured data as slots.
   * `--slot="VERIFICATION_LOG=$(cat /tmp/log)"`
   * `--slot="ALLOWLIST_CHECK=PASS"`
   * `--out="storage/handoff/${DP_ID}-RESULTS.md"`

Constraint: The binary must fail if any verification command exits non-zero, unless the DP explicitly marks it as `(allow-fail)`.

4.3 Component 3: The Integrity Linter
File: `tools/lint/integrity.sh`

This new tool serves as the "Semantic Guardrail" for scope.

Logic:
1. Read `Target Files allowlist` from `TASK.md`.
2. Read `git diff --name-only` (staged + unstaged) and `git ls-files --others --exclude-standard`.
3. **Assert:** `Set(Changed_Files) ⊆ Set(Allowlist)`.
4. **Fail Condition:** If any file is changed but not in the allowlist, exit `1` with a detailed error listing the unauthorized files.

4.4 Component 4: The Results Linter
File: `tools/lint/results.sh`

To ensure the `RESULTS.md` file itself hasn't been tampered with after generation.

Logic:
1. Parse `storage/handoff/*-RESULTS.md`.
2. Verify the file matches the `results.md.tpl` structure.
3. Verify the `GIT_HASH` in the receipt matches `git rev-parse HEAD`.
4. Verify the `Mandatory Closing Block` is populated and not placeholder text.

---

5. Implementation Plan

To deploy the System Locked Protocol, we will execute the following sequence.

Phase 1: Tooling (The Foundation)
1. **Create Template:** Author `ops/src/surfaces/results.md.tpl` with the strict schema.
2. **Create Integrity Linter:** Implement `tools/lint/integrity.sh` to enforce the allowlist logic currently described in `PoW.md` but not mechanically enforced.
3. **Create Certifier:** Implement `ops/bin/certify` to wrap the execution loop and template rendering.

Phase 2: Integration (The Cutover)
1. **Update PoT:** Amend `PoT.md` to require `ops/bin/certify` for all closeouts.
2. **Update Manual:** Rewrite `docs/MANUAL.md` "Closeout Cycle" to replace "Verify -> Copy/Paste" with "Run `ops/bin/certify`".
3. **Update Gates:** Add `tools/lint/results.sh` to `.github/workflows/repo_gates.yml`.

Phase 3: Validation (The Drill)
1. **Drift Test:** Attempt to generate a receipt with an out-of-scope file modification. Verify `certify` aborts.
2. **Hallucination Test:** Attempt to manually write a `RESULTS.md` file that deviates from the template. Verify `results.sh` rejects it.

---

6. Impact Analysis

6.1 Solving the "Process Integrity" Core Issue
* **Ambiguity:** The `certify` binary removes the ambiguity of "how to verify." It simply runs the script.
* **Stale Contamination:** Because `certify` runs in real-time and generates the artifact instantly, it is impossible to have a receipt that predates the code changes (assuming the commit happens immediately after).
* **Transaction Boundary:** The `RESULTS` generation becomes the *Prepare* phase of the commit transaction. If generation fails, the commit cannot proceed.

6.2 The "Contractor Efficiency Paradox"
Initially, Contractor efficiency may appear to drop. Agents will hit the `integrity.sh` hard stop frequently. This is **feature, not bug**. It forces the surfacing of hidden debt (scope creep) that was previously ignored. The system self-optimizes towards **Hyper-Literalism**, where `TASK.md` becomes a precise machine-executable specification.

6.3 Final Recommendation
The **Template-Based Certifier** is the correct architectural solution for Stela. It aligns with the existing investment in `ops/bin/template` and `ops/bin/draft`, completing the V-Model of governance. By treating the "Receipt" as a computed view of the "Work," rather than a user-authored document, we achieve cryptographic process integrity.

Next Step: Authorize the creation of the `ops/src/surfaces/results.md.tpl` and `ops/bin/certify` artifacts in the next Dispatch Packet.

Works cited

1. The Agent Integrity Framework: The New Standard for Securing Autonomous AI - Acuvity AI, accessed February 17, 2026, https://acuvity.ai/the-agent-integrity-framework-the-new-standard-for-securing-autonomous-ai/
2. FLASH: A Workflow Automation Agent for Diagnosing Recurring Incidents - Microsoft, accessed February 17, 2026, https://www.microsoft.com/en-us/research/wp-content/uploads/2024/10/FLASH_Paper.pdf